Huh. Somehow the files generated inside the container are here on our local filesystem. Aren't containers completely isolated? How did that happen?

The answer lies in the -v option we ignored in our docker run command. In Docker parlance, this *mounts a volume*—effectively sharing a portion of our local filesystem with the container. Specifically, -v ${PWD}:/usr/src/app says, "Mount our current directory inside the container at /usr/src/app" (${PWD} is a Unix environment variable pointing to the current directory). This means that any files in our local directory would be visible in /usr/src/app inside the container. Similarly, if we create, delete, or edit files in this container directory, the changes will be reflected on our local filesystem.

Here, mounting the local volume meant that the Rails project generated inside the container (within /usr/src/app) remained in our local directory, even after the container terminated. Additionally, this feature will be useful during development to allow us to edit files locally, and have the changes be picked up automatically inside the container, without having to rebuild the image.

It's worth noting a couple of key points about how this mounting behavior works. Firstly, if the /usr/src/app directory did not already exist inside the container, Docker would create it. Secondly, if the directory does exist inside the container, the mounted directory overlays and *masks* its contents while the mount is in effect.

Linux Users Only: Change File Ownership



You'll notice that our newly generated Rails project files are owned by root. That's because, by default, containers run as root (UID 1). In order to modify the files, you'll need to change their owner:

```
$ sudo chown <your_user>:<your_group> -R myapp/
```

You'll have to do this whenever we generate files inside a container. For more details, see File Ownership and Permissions, on page 199.

Finally, we come to the -i and -t options. To understand these, we first have to understand Docker's architecture.

The heart of Docker—the Docker Engine—is a client-server application. The Docker CLI (the docker command) is just a thin client that tells a separate program—the Docker daemon—to do what we've asked. The daemon is responsible for doing the heavy lifting in terms of starting, stopping, and otherwise bossing around our containers.